

Indian Institute of Technology Bombay

Department of Computer Science and Engineering

CS6008 : Network Security

DNS Security: Building, Attacking, and Defending a Hierarchical DNS

Assignment Report

Name	Reman Krishnan, Anant Kashid
Roll No.	26D0365, 26V0011
Course	CS6008: Network Security
Programme	Ph.D / B.Tech

September 2026

Contents

1 Overview and Methodology **3**
1.1 Environment and tooling . . . 3
1.2 Privilege model . . . 4
1.3 The laboratory topology . . . 5

2 Building the DNS Hierarchy (Phase 1) **6**
2.1 Objective . . . 6
2.2 Design and implementation . . . 6
2.3 Verification . . . 7
2.4 Tracing the referral chain . . . 9
2.5 Report answers . . . 10

3 On-Path Response Forgery (Phase 2) **11**
3.1 Objective . . . 11
3.2 Design and implementation . . . 11
3.3 Verifying the transparent relay . . . 11
3.4 The attack . . . 12
3.5 Report answers . . . 15

4 DNSSEC Chain of Trust (Phase 3) **16**
4.1 Objective . . . 16
4.2 Design and implementation . . . 16
4.3 Verifying a healthy signed hierarchy . . . 17
4.4 Re-running the forgery under DNSSEC . . . 18
4.5 Report answers . . . 19

5 Signature Stripping and Downgrade (Phase 4) **21**
5.1 Objective . . . 21
5.2 Design and implementation . . . 21
5.3 Verification . . . 21
5.4 Run A, stripping against the full chain . . . 22
5.5 Run B, the operator mistake . . . 23
5.6 Report answers . . . 25

6 Zone Walking (Phase 5) **26**
6.1 Objective . . . 26
6.2 Design and implementation . . . 26
6.3 Verifying how a signed zone denies a name . . . 26
6.4 Walking the zone . . . 27
6.5 Closing the leak with NSEC3 . . . 28
6.6 Report answers . . . 29

7 Last-Mile Response Forgery (Phase 6) **31**
7.1 Objective . . . 31
7.2 Design and implementation . . . 31
7.3 Verification . . . 31
7.4 The attack . . . 32
7.5 Report answers . . . 33

8 Encrypting the Last Mile with DoT and DoH (Phase 7) 35
8.1 Objective . . . 35
8.2 Design and implementation . . . 35
8.3 Verifying an encrypted, authenticated last mile . . . 36
8.4 Three attempts against two clients . . . 37
8.5 Report answers . . . 38

9 Synthesis and Conclusions 40

A Repository Layout and Reproduction 41

B Appendix A. Generative AI Usage Record 41
B.1 Verification and responsibility . . . 43

1 Overview and Methodology

DNS answers, in their original and still most common form, travel as unauthenticated cleartext. Anyone who sits on the path between a client and the nameservers it depends on can read a query as it passes and, just as easily, forge the reply that comes back. This single property, the absence of any proof that an answer came from the party entitled to give it, is the thread that runs through the whole assignment. Each phase of this report either exploits that property from a particular position or adds a mechanism intended to remove it at a particular layer.

The methodology is deliberately uniform across the seven phases, so that the results can be compared like for like. Every phase begins by deploying or extending the containerised hierarchy, then verifies that the deployment behaves as intended with the attacker absent or passive, and finally mounts the attack and records exactly what the resolver cached, what the client received, and what the packet capture showed. Keeping the structure constant means that any change in outcome from one phase to the next can be attributed to the one thing that changed between them, which is usually a single defensive mechanism or a single configuration choice. The attacker itself is kept as constant as possible and is simply relocated or extended, rather than rewritten, so that a defence is tested against the same adversary that defeated the layer before it.

A short note on terminology is useful before the details begin. The resolver is the recursive, caching server that the client asks. The authoritative servers are the root, the `com` top-level domain, and the `example.com` domain servers that actually hold the data. The attacker is a single container that is placed on whichever subnet the phase in question requires. All addresses are fixed in advance and are reused unchanged in every phase, which is what makes the captures directly comparable.

1.1 Environment and tooling

Every node in the laboratory is a Docker container with a fixed address on one of three isolated bridge networks. Isolation is a property of the topology rather than of any firewall rule. No subnet has a direct link to any other, so every packet that crosses from one subnet to another must pass through one of two routers, and those routers are the only containers permitted to forward traffic. This matters for the attacks that follow, because the adversary never has to defeat a filter. It only has to occupy a position that the topology already forces traffic through.

Component	Subnet	Address	Role
client	client	172.16.1.10	End host issuing lookups and web requests
webserver	client	172.16.1.20	Serves the legitimate `example.com` site over HTTP
rogue-web	client	172.16.1.30	Adversary's web server hosting the defacement page
resolver	resolver	172.16.2.10	Recursive caching resolver that validates DNSSEC from Phase 3
root-ns	authoritative	172.16.3.10	Authoritative server for the root zone (`.`)
tld-ns	authoritative	172.16.3.20	Authoritative server for the `.com` zone
domain-ns	authoritative	172.16.3.30	Authoritative server for `example.com`
edge-router	client/resolver	172.16.1.2 / 172.16.2.2	Routes client to resolver traffic, no NAT
core-router	resolver/auth	172.16.2.3 / 172.16.3.3	Routes resolver to authoritative traffic, no NAT
attacker	resolver (P2–5)	172.16.2.100	On-path adversary on the resolver subnet
attacker	client (P6–7)	172.16.1.100	The same adversary, moved to the client's last mile

Table 1: Fixed assignment of containers to subnets and addresses, used unchanged across all phases.

The DNS daemon on every nameserver and on the resolver is BIND, confirmed at version 9.20.x on each node. The evidence for Phases 1 to 4 was captured with the package 9.20.30, and a few of the later phases with 9.20.23 from Debian backports, a difference that reflects only when those container images happened to be rebuilt. Both are BIND 9.20.x, and the point-release gap does not affect any behaviour observed in this report. Inspection of results uses `dig`, `delv`, `kdig`, and `curl`, with `tcpdump` and `tshark` for packet capture. The private certificate authority and the resolver's TLS certificate are produced with OpenSSL. Routing is plain Linux IP forwarding through `iproute2`, with static routes on every node and no network address translation anywhere, so that addresses stay honest from end to end and a capture shows the real source and destination of every packet.

A constraint worth stating at the outset is that every attack program was written in C using `libpcap`, which both captures packets off the wire and injects crafted ones back onto it through `pcap_sendpacket`. High-level spoofing or DNS frameworks were not used for the attacks. This keeps the adversary honest about what an on-path attacker actually has to do, namely parse raw frames, rewrite only the fields it cares about, recompute the IP and UDP checksums, and inject the result, rather than hiding that machinery behind a library call. The inspection tools named above are used only to verify results.

1.2 Privilege model

The privilege policy follows the principle of least authority and underpins the isolation claim. The two routers carry `NET_ADMIN` and are the only nodes with IP forwarding enabled. They hold the routing table entries that join the subnets and, from Phase 2 onward, the diversion routes that steer traffic through the attacker. Every other node receives `NET_ADMIN` only so that it can install the static routes it needs to reach off-subnet addresses through its local router, and none of them forward, bridge, or translate. The attacker additionally holds `NET_RAW`, which is what `libpcap` requires to capture and inject at the link layer. Nothing in the laboratory is granted more capability than its role demands.

1.3 The laboratory topology

Figure 1 shows the full deployment. The three coloured regions are the three bridge networks. The routers straddle two networks each and are drawn on the boundary between them, which is exactly where they sit in the address plan. The attacker is shown in both of its positions, on the resolver subnet for Phases 2 to 5 and on the client subnet for Phases 6 and 7, because the same adversary is simply relocated as the defences push the interesting attack surface closer to the client.

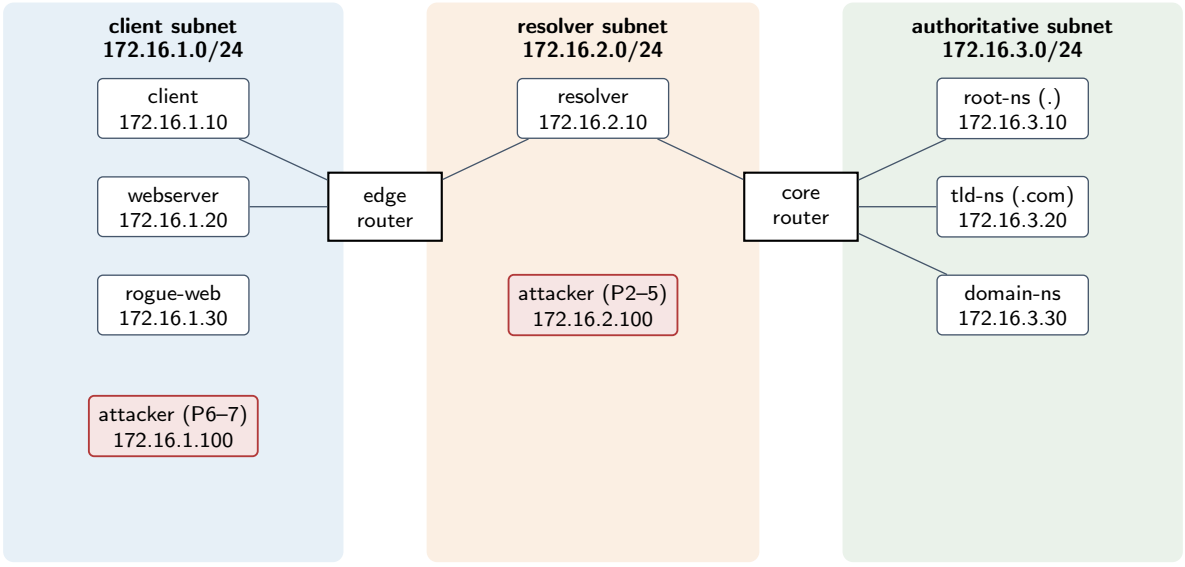


Figure 1: The laboratory topology. Three isolated bridge networks are joined only by two forwarding routers. The adversary is the same container in two positions, on the resolver subnet for Phases 2 to 5 and relocated to the client's last mile for Phases 6 and 7.

A note on reproducibility applies to the whole submission. Each phase directory contains its own Compose file, zone and daemon configuration, router and diversion scripts, attack source with a build command, and a README listing the exact commands to deploy, attack, and verify. A grader on a clean checkout, with only the host tools listed above, can reproduce the results from that README alone, because the attack programs discover the resolver's hardware address and the relevant interface at run time rather than hard-coding them. Compiled binaries, signed zones, generated keys, and built images are regenerated from source and are kept out of the committed tree.

2 Building the DNS Hierarchy (Phase 1)

2.1 Objective

The first phase has no attacker. Its purpose is to build the baseline that every later phase extends and to prove two things about it. The first is that it resolves names correctly through a genuine iterative lookup. The second is that its subnets are isolated, so that the only way between them is through a router. Getting this right is what makes the later attacks meaningful, because an attack is only interesting if the thing it breaks was working properly to begin with.

2.2 Design and implementation

The hierarchy is the textbook three tiers. The root nameserver delegates `com.` to the TLD server, the TLD server delegates `example.com.` to the domain server, and the domain server holds the A records for `example.com` and `www.example.com`, both pointing at the web server at 172.16.1.20. The resolver is recursive, and in this phase DNSSEC validation is deliberately left off, because the whole point of Phase 1 is to establish the unprotected baseline. The zone file for `example.com` is small and readable.

Listing 1: The example.com zone served by domain-ns in Phase 1 (zones/db.example.com).

```
$ORIGIN example.com.
$TTL 300
@   IN SOA ns.example.com. hostmaster.example.com. (
        2026102201 3600 600 86400 300 )
@   IN NS  ns.example.com.
ns  IN A   172.16.3.30
@   IN A   172.16.1.20
www IN A   172.16.1.20
```

Reachability is arranged entirely with static routes and IP forwarding, and no NAT is used anywhere, so that the addresses a capture shows are the real ones. Table 2 lists the route that each node actually carries, taken from the deployment. The pattern is simple to state. Every non-router node has a connected route for its own subnet and one static route for each off-subnet destination it needs, pointed at the router on its segment, while the two routers have only their two connected subnets and are the sole nodes with forwarding enabled.

Node	Static route(s) installed	IP forwarding
client	172.16.2.0/24 via 172.16.1.2 (edge)	off
resolver	172.16.1.0/24 via 172.16.2.2 (edge), and 172.16.3.0/24 via 172.16.2.3 (core)	off
root-ns	172.16.2.0/24 via 172.16.3.3 (core)	off
tld-ns	172.16.2.0/24 via 172.16.3.3 (core)	off
domain-ns	172.16.2.0/24 via 172.16.3.3 (core)	off
edge-router	connected to 172.16.1.0/24 and 172.16.2.0/24	**on**
core-router	connected to 172.16.2.0/24 and 172.16.3.0/24	**on**

Table 2: Static routes and forwarding state per node, from the deployed hierarchy. Only the two routers forward, which is what makes the subnets reachable only through them.

The client is configured to use the resolver as its only nameserver, through a `resolv.conf` that names 172.16.2.10 with a short timeout, so that every lookup the client makes is a recursive query to the resolver rather than a direct query to any authoritative server. The root hints and the glue records name the nameservers at their true 172.16.3.x addresses, so the resolver performs a genuine iterative walk.

2.3 Verification

Three checks establish that the baseline works. First, the BIND version is confirmed on each server, so that later behaviour can be attributed to configuration rather than to a version quirk.

Listing 2: BIND version confirmed on every server (evidence/bind-versions.txt).

```
--- resolver / root-ns / tld-ns / domain-ns ---
BIND 9.20.30-1~deb13u1-Debian (Stable Release)
```

Second, from the client, `dig example.com` returns 172.16.1.20 and `curl http://example.com` returns the real page, which together prove end-to-end resolution and fetch. The same holds for `www.example.com`.

```
### phase1/evidence/example-com.txt


; <<>> DiG 9.20.30-1~deb13u1-Debian <<>> example.com A +time=3 +tries=1
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 60115
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: ab8230b2aed31885010000006ad98e6b6ba6889bb901d8d5 (good)
;; QUESTION SECTION:
;example.com.   IN A

;; ANSWER SECTION:
example.com.  167 IN A 172.16.1.20

;; AUTHORITY SECTION:
example.com.  167 IN NS ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.  167 IN A 172.16.3.30

;; Query time: 0 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 04:17:47 UTC 2026
;; MSG SIZE  rcvd: 117


### phase1/evidence/www-example-com.txt


; <<>> DiG 9.20.30-1~deb13u1-Debian <<>> www.example.com A +time=3 +tries=1
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 58189
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 334a5d6c0b7d048b010000006ad98d9024f4403b0550b7ed (good)
;; QUESTION SECTION:
;www.example.com.  IN A

;; ANSWER SECTION:
www.example.com. 300 IN A 172.16.1.20

;; AUTHORITY SECTION:
example.com.  300 IN NS ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.  300 IN A 172.16.3.30

;; Query time: 3 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 04:14:08 UTC 2026
;; MSG SIZE  rcvd: 121
```

LEGITIMATE EXAMPLE.COM WEBSITE

CS6008 DNS Security Assignment — Phase 1

Expected server address: 172.16.1.20

Figure 2: Client-side resolution of `example.com` and `www.example.com`, both returning 172.16.1.20, and the `curl` fetch of the legitimate page.

The screenshot in Figure 2 is the evidence for requirement 1.2 that the names resolve to the web server and that the fetched page is the genuine one. The reader should look for the answer section giving 172.16.1.20 and for the page body identifying itself as the legitimate site served from that address, which together confirm that the client reaches the real web server through normal recursive resolution.

Third, a query sent directly from the client to the root server at 172.16.3.10 gets no answer, because the client has no route into the authoritative subnet and the routers do not carry it there. This is the isolation check.

```
### phase1/evidence/direct-root-query.txt

;; UDP setup with 172.16.3.10#53(172.16.3.10) for example.com failed: network unreachable.
;; no servers could be reached
```

Figure 3: The isolation check. A query sent directly from the client to root-ns at 172.16.3.10 receives no answer because the authoritative subnet is not directly reachable.

The screenshot in Figure 3 demonstrates requirement 1.2's isolation condition. The timeout, rather than any answer, is the point. It shows that the client cannot talk to an authoritative server directly, so the only way its lookups can succeed is through the resolver, and the only way the resolver can reach the authoritative subnet is across the core router.

2.4 Tracing the referral chain

The capture on the core router is the most informative piece of Phase 1 evidence, because it shows the resolver doing the iterative walk by hand. Rather than reproduce a raw packet dump, Table 3 summarises the relevant messages from the submitted capture, drawn directly from the packet summary. The structure is unmistakable. There is a query to the root that is answered by a referral to `com`, then a query to the TLD that is answered by a referral to `example.com`, and finally a query to the domain server that returns the address.

#	Source	Dest	Txn ID	Type	Meaning
1	172.16.2.10	172.16.3.10	0x2493	query	resolver asks root for `example.com` A
3	172.16.3.10	172.16.2.10	0x2493	reply	root refers to `com`, names ns at 172.16.3.20
5	172.16.2.10	172.16.3.20	0x7140	query	resolver asks `com` TLD for `example.com` A
6	172.16.3.20	172.16.2.10	0x7140	reply	TLD refers to `example.com`, names ns at 172.16.3.30
7	172.16.2.10	172.16.3.30	0xa9a8	query	resolver asks domain server for `example.com` A
8	172.16.3.30	172.16.2.10	0xa9a8	reply	authoritative answer, `example.com` is 172.16.1.20

Table 3: The Root to TLD to Domain referral chain, reconstructed from the Phase 1 core-router capture. Frames 2 and 4 (omitted) are a separate root NS priming exchange. Transaction IDs and ports are specific to this capture.

```
### tshark -r phase1/evidence/core-dns.pcap -Y 'dns'

   (open phase1/evidence/core-dns.pcap in the Wireshark GUI and screenshot - see SCREENSHOTS.md)
```

Figure 4: Annotated `tshark` decode of the referral chain captured on the core router, with the root, com, and example.com exchanges marked.

The screenshot in Figure 4 is the capture that requirement 1.3 asks for. It should show, in order, the query to the root and its referral to `com`, the query to the TLD and its referral to `example.com`, and the final query to the domain server and its address answer. The matching transaction identifiers between each query and its reply are worth marking, because they are exactly the values that make a reply acceptable to the resolver, and they are sitting in plain view.

2.5 Report answers

What the capture exposes on the wire. Every field that matters is in plain view. Each message carries the query name, the 16-bit transaction identifier, the UDP source port the resolver chose, and, in the responses, the referral or the final answer with its addresses. Nothing is encrypted and nothing is signed. An observer on this path learns not only what is being asked but also the two values, the transaction ID and the source port, that a forged reply would need to match in order to be accepted.

How the resolver's query reaches root-ns. The capture shows the resolver's packets leaving with source address 172.16.2.10 and arriving at 172.16.3.10, and the resolver's routing table sends traffic for the authoritative subnet to next hop 172.16.2.3, which is the core router. The relevant route, `172.16.3.10 via 172.16.2.3 dev eth0 src 172.16.2.10`, is shown directly in the evidence. The query therefore crosses exactly one router on its way to the root, and that router is a node the experimenter controls.

What a node on the resolver's path could do. Because the exchange is cleartext and the resolver accepts the first well-formed reply that matches the query name, transaction ID, and port, any node the traffic passes through can read those values and manufacture a reply that carries them. On this topology the core router, or anything the core router routes traffic through, is precisely such a node. This is the capability that the response-forgery attack relies on.

Key observation. The baseline resolves correctly and the subnets are genuinely isolated, yet the resolution itself is completely exposed. Every identifier that an attacker would need in order to forge an acceptable reply is visible to any node on the resolver's path.

3 On-Path Response Forgery (Phase 2)

3.1 Objective

Phase 2 turns the Phase 1 observation into a working attack. An adversary is placed on the resolver's subnet at 172.16.2.100, and the goal is to poison the resolver's cache for a single name, `example.com`, so that the client's web traffic is redirected to the rogue server, while every other name, including `www.example.com`, continues to resolve correctly. The surgical nature of the goal is important, because an attack that breaks everything is easy to detect, whereas an attack that quietly redirects one name is the realistic threat.

3.2 Design and implementation

Two design decisions define the attack. The first is how the attacker gets onto the path. Rather than ARP spoofing or any change at the resolver, the diversion is a single route added on the core router that sends the resolver's inbound reply traffic through the attacker, which then relays it onward. The resolver and the nameservers keep their Phase 1 configuration untouched, which is what makes the demonstration clean, because the attack lives entirely in a position the topology already grants.

Listing 3: The Phase 2 diversion on the core router, which finds the resolver-facing interface by its address so nothing is hard-coded (scripts/divert-on.sh). The resolver's own configuration is never touched.

```
DEV=$(ip -o -4 addr show | awk '$4 ~ /^172\.16\.2\./{print $2; exit}')
ip route replace 172.16.2.10 via 172.16.2.100 dev "$DEV"
```

The second decision is the interceptor itself, in `src/interceptor.c`, written in C with `libpcap`. It opens a live capture on its interface with the Berkeley Packet Filter `udp and port 53`, so it sees only DNS traffic, and it runs in one of two modes. In relay mode, and for any name that is not the target in forge mode, it takes the captured frame, rewrites the link-layer source and destination so the frame is delivered on to the resolver, recomputes the IP and UDP checksums, and injects it unchanged with `pcap_sendpacket`. This is what keeps non-target resolution working.

For the target name in forge mode, the interceptor does not relay the genuine reply at all. It constructs a fresh DNS response that copies the question section and the transaction identifier from the real exchange, sets the standard query response flags, and appends a single answer record of type A with a 300-second time to live whose address is the rogue server at 172.16.1.30. It then builds the IP and UDP headers, computes both checksums, and injects the forged frame to the resolver. Because the diversion makes the attacker the sole relay for resolver-bound replies, the resolver never sees the genuine answer for the target name. It sees only the forgery. That is how the forged answer is accepted over the genuine one, without any race or timing trick. The interceptor also reads its own hardware address at start-up rather than assuming it.

3.3 Verifying the transparent relay

Before forging anything, the interceptor is run in pure relay mode to prove that merely being on the path does not disturb resolution. With the diversion active and the relay running, the client still resolves `example.com` to 172.16.1.20 and fetches the real page. This isolates the effect of the forgery in the next step from any effect of the diversion itself.

```
### phase2/evidence/p2-relay-dig.txt


; <<>> DiG 9.20.30-1~deb13u1-Debian <<>> example.com A +time=3 +tries=1
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 9412
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: ef3ae1e93088a3b1010000006ad99a5311a257ef62fe426f (good)
;; QUESTION SECTION:
;example.com.   IN A

;; ANSWER SECTION:
example.com.  300 IN A 172.16.1.20

;; AUTHORITY SECTION:
example.com.  300 IN NS ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.  300 IN A 172.16.3.30

;; Query time: 4 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 05:08:35 UTC 2026
;; MSG SIZE  rcvd: 117


### phase2/evidence/p2-relay-curl.txt

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>CS6008 Legitimate Website</title>
</head>
<body>
  <h1>LEGITIMATE EXAMPLE.COM WEBSITE</h1>
  <p>CS6008 DNS Security Assignment — Phase 1</p>
  <p>Expected server address: 172.16.1.20</p>
</body>
</html>
```

Figure 5: Transparent relay mode. With the attacker on the path but forging disabled, `example.com` still resolves to 172.16.1.20 and the real page is fetched.

The screenshot in Figure 5 is the verification required by 2.2. It should show the client still receiving 172.16.1.20 and the genuine page while the attacker is actively relaying, which establishes that the diversion by itself changes nothing. Any change seen once forging is enabled is therefore the work of the forgery alone.

3.4 The attack

With forging enabled and the resolver cache flushed, the client's lookup of `example.com` now returns 172.16.1.30, and the fetch returns the rogue defacement page. The captured `dig` output

below is the real result from the submitted evidence.

Listing 4: The poisoned answer. example.com now resolves to the rogue server at 172.16.1.30 (evidence/p2-attack-dig.txt).

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13528
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2
;; ANSWER SECTION:
example.com.        300  IN  A  172.16.1.30
```

The non-target check demonstrates the surgical quality of the attack. At the same time, `www.example.com` still resolves to 172.16.1.20, because the interceptor rewrites only answers whose query name is exactly `example.com` and relays everything else byte for byte.

Listing 5: The non-target name is untouched. www.example.com still resolves to 172.16.1.20 (evidence/p2-nontarget-dig.txt).

```
;; ANSWER SECTION:
www.example.com.    300  IN  A  172.16.1.20
```

```
### phase2/evidence/p2-attack-dig.txt


; <<>> DiG 9.20.30-1~deb13u1-Debian <<>> example.com A +time=3 +tries=1
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 31219
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 93e3843404b9398e010000006ad99a65259473144c53af9c (good)
;; QUESTION SECTION:
;example.com.   IN A

;; ANSWER SECTION:
example.com.  300 IN A 172.16.1.30

;; AUTHORITY SECTION:
.   300 IN NS ns.root.

;; Query time: 1 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 05:08:53 UTC 2026
;; MSG SIZE  rcvd: 104


### phase2/evidence/p2-nontarget-dig.txt


; <<>> DiG 9.20.30-1~deb13u1-Debian <<>> www.example.com A +time=3 +tries=1
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 31139
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: bb8adca52ae20aba010000006ad99a656fd9088ed417d14e (good)
;; QUESTION SECTION:
;www.example.com.  IN A

;; ANSWER SECTION:
www.example.com. 300 IN A 172.16.1.20

;; AUTHORITY SECTION:
example.com.  300 IN NS ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.  300 IN A 172.16.3.30

;; Query time: 1 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 05:08:53 UTC 2026
;; MSG SIZE  rcvd: 121
```

DEFACED / HIJACKED EXAMPLE.COM WEBSITE

CS6008 DNS Security Assignment — Phase 2 On-Path Poisoning

Rogue server address: 172.16.1.30

Figure 6: The poisoning result. The client resolves `example.com` to 172.16.1.30 and `curl` returns the rogue defacement page, while `www.example.com` still returns 172.16.1.20.

The screenshot in Figure 6 is the central evidence for 2.3 and 2.4. It should show three things together. The target name now resolves to the rogue address, the rogue page actually loads in place of the real one, and the non-target name still resolves correctly. The contrast between the first two and the third is the proof that the poisoning is precise rather than a blanket interception.

```
### tshark -r phase2/evidence/p2-attack.pcap -Y 'dns'

   (open phase2/evidence/p2-attack.pcap in the Wireshark GUI and screenshot - see SCREENSHOTS.md)

### phase2/evidence/interceptor-forge.log

Starting DNS Interceptor [FORGE mode] on eth0...
Attacker MAC: a6:07:a7:2c:96:2a | Target Resolver MAC: 62:dc:c3:52:ce:78
[FORGE] Poisoning A record for example.com (ID 0xf1f3) -> 172.16.1.30
[RELAY] Relaying message for <non-query> (ID 0xd5ba)
[RELAY] Relaying message for example.com (ID 0x3a24)
[RELAY] Relaying message for <non-query> (ID 0xae14)
[RELAY] Relaying message for example.com (ID 0x3e79)
[RELAY] Relaying message for example.com (ID 0x7be0)
[RELAY] Relaying message for www.example.com (ID 0x6dbe)
[RELAY] Relaying message for www.example.com (ID 0x79ac)
```

Figure 7: Annotated capture of the interception and injection on the core router, showing the genuine reply from the authoritative server and the forged reply carrying 172.16.1.30.

The screenshot in Figure 7 is the packet-level evidence 2.4 asks for. It should show the forged response carrying 172.16.1.30 reaching the resolver, and the interceptor's own log line recording that it poisoned the answer for `example.com`. Reading the capture together with the log confirms that the rogue address the client received originated from the attacker's injection and not from any authoritative server.

3.5 Report answers

Why the attack succeeds. Plain DNS has no way for a resolver to tell a genuine answer from a forged one. A UDP response is accepted if it arrives on the right socket and carries the matching query name, transaction ID, and source port, and nothing in the protocol binds those fields to the party actually entitled to answer. The on-path position converts this from a guessing game into a certainty. An off-path attacker would have to guess the transaction ID and the resolver's ephemeral source port, which together give a large search space, whereas the on-path attacker simply reads both out of the query as it passes and writes them into the forgery. That is the whole of the advantage, and it is decisive.

How non-target names are left untouched. The interceptor keys its rewriting on the query name in the DNS question section. For any packet whose question is not exactly `example.com` it performs no edit and injects the original bytes, with only the link-layer header and checksums adjusted so the frame forwards correctly, so `www.example.com` and every other name follow their normal path to the genuine answer. The forgery is therefore a precise substitution for one name rather than a blanket interception.

Outcome. The attack succeeds. Plain DNS trusts the wire, and an on-path attacker owns the wire. One name is redirected to a hostile server while the rest of resolution carries on normally, which is exactly the stealthy outcome that makes this threat realistic.

4 DNSSEC Chain of Trust (Phase 3)

4.1 Objective

Phase 3 adds the defence that directly answers the forgery of Phase 2, which is a cryptographic chain of trust over the data itself. The idea is to sign every zone so that a resolver can verify, from a single trust anchor it holds in advance, that an answer really came from the zone that owns the name and was not altered on the way. The attacker and its Phase 2 binary are left completely unchanged. Only the zones and the resolver change, so that any difference in outcome is attributable to DNSSEC and to nothing else.

4.2 Design and implementation

Each authoritative server signs its own zone. The root signs the root, the TLD signs `com`, and the domain server signs `example.com`. The zones are then chained with delegation signer, or DS, records placed in each parent, so that `com`'s DS is published in the root and `example.com`'s DS is published in `com`. This produces an unbroken chain from the root key down to the address record the client wants. Figure 8 shows the structure.

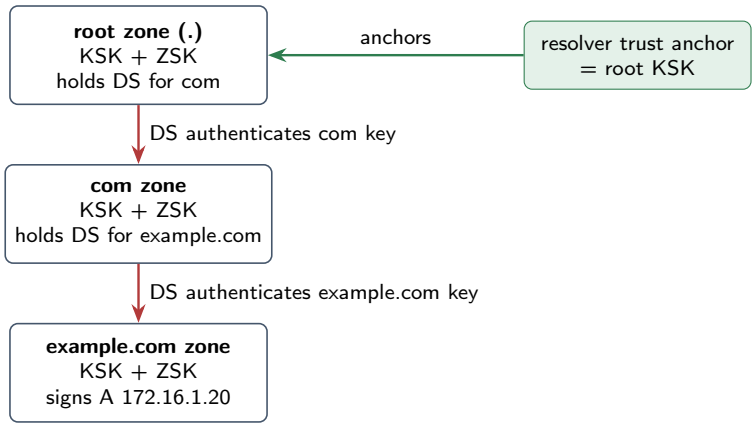


Figure 8: The DNSSEC chain of trust. The resolver trusts only the root key signing key, and each parent's signed DS record authenticates the child's key, so trust flows from the root down to the example.com A record without the resolver needing to know any key but the anchor.

The signing is done by `scripts/sign-zones.sh`, which for each of the three zones generates a zone signing key and a key signing key with `dnssec-keygen`, appends both public keys to the zone, and signs the zone with `dnssec-signzone`. It then derives the child's DS record from its key signing key with `dnssec-dsfromkey -2`, which uses SHA-256 for the digest, and appends that DS into the parent zone before signing the parent. The order is bottom up, that is `example.com` first, then `com` with `example.com`'s DS, then the root with `com`'s DS, which is what produces a consistent chain. Finally the script extracts the root key signing key and writes it into the resolver's trust anchor file.

Listing 6: The heart of the signing script, which generates keys, publishes the child DS in the parent, and signs bottom up (scripts/sign-zones.sh, abridged).

```
dnssec-keygen -a ECDSAP256SHA256 -n ZONE example.com        # ZSK
dnssec-keygen -a ECDSAP256SHA256 -f KSK -n ZONE example.com # KSK
dnssec-signzone -S -o example.com -f db.example.com.signed db.example.com.unsigned
dnssec-dsfromkey -2 Kexample.com.+013+XXXXX.key > dsset-example.com
# dsset-example.com is appended into com before com is signed; com's DS into root.
```

On the resolver side, validation is switched on with `dnssec-validation yes` and the root key signing key is installed as the single trust anchor, while the root hints are left pointing at the lab root server exactly as in Phase 1.

Listing 7: The resolver's trust anchor, a static root KSK (configs/trust-anchors.conf).

```
trust-anchors {
    . static-key 257 3 13 "A+jYngeG3x3mfMvQKa3yfi41+fM/f9/Z/GapyPr+Ztd...==";
};
```

Signing algorithm choice. The zones are signed with ECDSA using the P-256 curve and SHA-256, which is algorithm 13. This is a deliberate choice. Compared with RSA it produces much smaller keys and signatures for equivalent strength, which keeps DNS responses compact and reduces the chance of fragmentation or fallback to TCP, and it is widely supported by modern validating resolvers. Its trade-off is that it is more sensitive to a sound source of randomness during signing and is slightly more expensive to verify than to generate, though neither matters at laboratory scale. The main practical cost is simply that very old resolvers might not recognise the algorithm, which is not a concern here.

4.3 Verifying a healthy signed hierarchy

Before attacking, the signed hierarchy is checked with no interceptor on the path. A `dig +dnssec example.com` returns the address together with its RRSIG and, most importantly, with the `ad` flag set, which is the resolver's statement that it validated the answer. An independent check with `delv`, pointed at the lab root key as its trust anchor, reports the answer as fully validated.

Listing 8: Healthy validation before the attack, showing the ad flag and a valid RRSIG, with delv reporting full validation (evidence/p3-valid-dig.txt, p3-valid-delv.txt).

```
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, ...
example.com.  300  IN  A      172.16.1.20
example.com.  300  IN  RRSIG  A 13 2 300 20261122104556 20261023104556 7712 example.com. ...

; fully validated          <-- delv, anchored on the lab root key
```

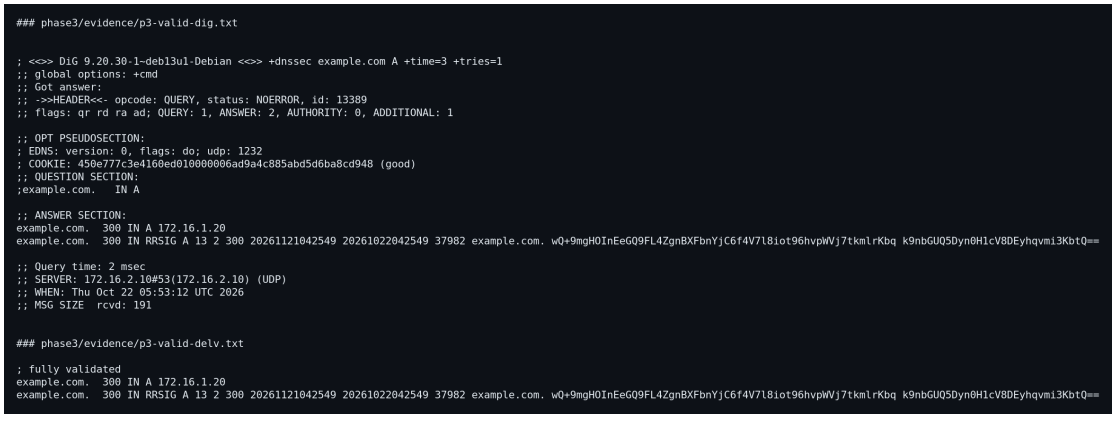


Figure 9: Healthy DNSSEC validation. `dig +dnssec` shows the `ad` flag and a valid RRSIG, and `delv` reports the chain as fully validated against the root trust anchor.

The screenshot in Figure 9 is the verification required by 3.2. It should show the `ad` flag in the `dig` header and the RRSIG accompanying the address, together with the `delv` line reporting full validation. These are the signs that the signed hierarchy is healthy and that the whole chain

from the root down to `example.com` verifies, which is the necessary starting point before the attack is re-run.

4.4 Re-running the forgery under DNSSEC

The Phase 2 attack is now re-run without modification, and the result is completely different. The forged and unsigned answer carries no valid signature from the `example.com` key, so the validating resolver refuses to accept it and fails the lookup rather than caching a lie. It exhausts its attempts to obtain a signed answer that it can validate and then returns SERVFAIL with an empty answer section, which the capture below records. The client's web fetch therefore fails at the name resolution step and never reaches the rogue server.

The cause of the failure is signature validation rather than any network fault. The attacker holds no zone key, so the record it injects carries no RRSIG that the resolver can verify against the `example.com` zone key, and a validating resolver treats a signed zone whose data does not verify as bogus. The very packet that poisoned the cache in Phase 2 is simply discarded here, which is the whole difference that DNSSEC makes.

Listing 9: The validating resolver returns SERVFAIL with an empty answer section for the forged answer (evidence/p3-attack-dig-servfail.txt).

```
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 60952
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1
;; QUESTION SECTION:
;example.com.                    IN      A
;; Query time: 4 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
```

```
### phase3/evidence/p3-attack-dig-servfail.txt

;; communications error to 172.16.2.10#53: timed out

; <<>> DiG 9.20.30-1~deb13u1-Debian <<>> example.com A +time=6 +tries=2
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 45196
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 9fd84e0353b7f984010000006ad9a607c0a3451938cfe417 (good)
; EDE: 22 (No Reachable Authority)
;; QUESTION SECTION:
;example.com.   IN A

;; Query time: 3996 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 05:58:31 UTC 2026
;; MSG SIZE  rcvd: 74


### phase3/evidence/p3-attack-curl.txt

  Total    Received Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
```

Figure 10: The re-run attack under DNSSEC. The validating resolver rejects the forged answer and returns SERVFAIL with an empty answer section, and `curl` cannot resolve the host, so the client never reaches the rogue page.

The screenshot in Figure 10 is the evidence for 3.3 and 3.4. It should show the resolver failing the lookup with a SERVFAIL status rather than returning the rogue address, and it should show the client's fetch failing to resolve the host. Together they establish that the identical attack which succeeded in Phase 2 no longer works once the resolver validates the chain, because the forged record carries no signature the resolver can verify.

4.5 Report answers

What the resolver checks, and how this differs from Phase 2. In Phase 2 the resolver accepted the first matching reply and cached 172.16.1.30. Here the resolver requires that the address record be accompanied by an RRSIG that verifies under the `example.com` zone key, that the zone key in turn be authenticated by the DS record in `com`, and that the `com` key be authenticated by the DS in the root, which is finally vouched for by the trust anchor the resolver holds. The forged answer satisfies none of this, because the attacker holds no zone key and cannot produce a valid signature, so the chain breaks and the resolver returns failure instead of a poisoned record.

What the attacker can still read, and what DNSSEC does and does not protect. Comparing the attacked exchange with the Phase 1 capture shows that DNSSEC changed nothing about confidentiality. The query name, the transaction identifier, the ports, and the answers are all still in cleartext on the wire, and the on-path attacker can read every one of them exactly as before. What DNSSEC adds is integrity and origin authentication of the data. The attacker can still see and still inject, but it can no longer make the resolver believe a forged record, because it cannot forge the signature. DNSSEC protects the authenticity of DNS data,

and it does not provide any privacy for it.

Whether any private key belongs in the repository. No. The private components of the key signing and zone signing keys are exactly what an attacker would need in order to forge valid signatures and defeat the entire chain, so they must never be committed. The repository carries the signing scripts and the public material needed to reproduce a signed hierarchy, and the keys themselves are regenerated on a clean checkout rather than shipped.

Outcome. The attack is blocked. DNSSEC authenticates the data, so the plain forgery that worked in Phase 2 now fails. The resolver detects the missing signature and refuses the answer. The attacker can still read everything on the wire, since DNSSEC adds no confidentiality.

5 Signature Stripping and Downgrade (Phase 4)

5.1 Objective

Since a forged signed answer failed in Phase 3, Phase 4 asks the obvious next question. Rather than forge a signature, can the attacker make the signed zone appear to be unsigned, so that the resolver stops expecting signatures and the plain forgery is accepted again? The attacker is allowed to rewrite anything in transit but still holds no private keys. The phase is run in two contrasting configurations, and the contrast between them is the entire lesson.

5.2 Design and implementation

The interceptor is extended into a stripper, in `src/stripper.c`, with three modes selected on the command line. The relay mode is the transparent relay used for verification. The two strip modes both forge the final `example.com` A answer, but they differ in how much of the DNSSEC material around it they remove.

In `-strip-runA`, the stripper removes every DNSSEC-related record type, namely RRSIG, DNSKEY, DS, NSEC, NSEC3, NSEC3PARAM, CDS, and CDNSKEY, from all sections of every response it relays. This is the fuller downgrade, because it removes not only the signatures on the `example.com` answer but also the DS record, and its RRSIG, from the `com` referral, which is the attacker's attempt to hide the fact that the child is supposed to be secure. In `-strip-runB`, the stripper forges the same `example.com` A answer but leaves the parent's records untouched. The attacker discovers the resolver's hardware address by ARP at start-up and detects its interface by address, so the behaviour does not depend on hard-coded values.

The two runs also differ in the deployment. Run A uses the unchanged Phase 3 hierarchy, in which `com` publishes a DS for `example.com`. Run B re-signs `com` with `scripts/sign-zones-runB.sh` so that it publishes no DS for `example.com`, while `example.com` itself stays fully signed and the root and the trust anchor are untouched. This single change models a specific and realistic operator error.

5.3 Verification

Before stripping, the client still resolves `example.com` as an authenticated answer through the transparent relay, with the `ad` flag set and the real address returned. This confirms that the starting point for both runs is a healthy, validated resolution and that the stripper in relay mode changes nothing.



Figure 11: Verification before stripping. Through the transparent relay the client still resolves `example.com` to 172.16.1.20 with the `ad` flag set and fetches the real page.

The screenshot in Figure 11 is the verification required by 4.2. It should show the authenti-

cated answer and the real page while the stripper is on the path in relay mode, which establishes that anything different seen in the two attack runs is caused by stripping and forging, not by the attacker's presence.

5.4 Run A, stripping against the full chain

In Run A the stripper removes every `example.com`-scoped DNSSEC record on the wire and injects the forged address, and its own log confirms that it did so. The resolver, however, returns SERVFAIL, and the fetch fails.

Listing 10: Run A result: the strip-and-forge attack fails (evidence/a-attack-dig.txt).

```
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 8968
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, ...
```

The reason is the most important idea in the phase. Because `com` publishes a DS for `example.com`, the resolver has already learned, from authenticated data, that `example.com` is supposed to be secure. A validating resolver only treats a delegation as insecure when it holds an authenticated proof that no DS exists, namely a signed denial from the parent. The on-path attacker holds no `com` key, so it can neither forge that proof nor strip its way to one. Removing the signatures does not make the zone look unsigned. It makes the zone look broken, and broken resolves to SERVFAIL rather than to a cached forgery. The resolver's log naming a broken trust chain is the direct evidence of this.

```
### phase4/evidence/stripper-runA.log

stripper: mode=--strip-runA dev=eth0 attacker=22:85:2a:fa:73:1f
stripper: resolver 172.16.2.10 is at f2:c7:38:e7:e1:3a
stripper: running
[STRIP] removed example.com DNSSEC RRs from reply (qname=example.com qtype=1) 248->101 bytes
[FORGE] example.com A (id 0x21f3) -> 172.16.1.30  [runA]
[STRIP] removed example.com DNSSEC RRs from reply (qname=example.com qtype=43) 215->68 bytes
[STRIP] removed example.com DNSSEC RRs from reply (qname=example.com qtype=28) 369->118 bytes
[STRIP] removed example.com DNSSEC RRs from reply (qname=example.com qtype=43) 215->68 bytes

### phase4/evidence/a-attack-dig.txt


; <<>> DiG 9.20.30-1~deb13u1-Debian <<>> example.com A +time=6 +tries=2
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 8968
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 602976584563d3ec010000006ad9c24b3c1b013362646302 (good)
;; QUESTION SECTION:
;example.com.   IN A

;; Query time: 36 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 07:59:07 UTC 2026
;; MSG SIZE  rcvd: 68


### phase4/evidence/a-resolver-log.txt

resolver-1  | 22-Oct-2026 07:59:07.723 broken trust chain resolving 'example.com/A/IN': 172.16.3.30#53
resolver-1  | 22-Oct-2026 07:59:07.833 broken trust chain resolving 'example.com/AAAA/IN': 172.16.3.30#53
```

Figure 12: Run A evidence. The stripper log shows the DNSSEC records removed and the A record forged, yet the resolver reports a broken trust chain and returns SERVFAIL.

The screenshot in Figure 12 is the Run A evidence for 4.3 and 4.4. It should show that the attacker genuinely stripped and forged, from its own log, and that the resolver nonetheless refused the result with a broken trust chain. The pairing matters, because it proves the failure

is the resolver detecting an unverifiable delegation, not the attack failing to execute.

5.5 Run B, the operator mistake

Run B changes one thing in the deployment: `com` is re-signed so that it no longer publishes `example.com`'s DS, while `example.com` itself stays fully signed and the root and the trust anchor are untouched. The attacker performs the same core strip-and-forge of the `example.com` answer as in Run A, but it now leaves the parent's records in place rather than stripping them. The reason is specific. In this deployment `com`'s own signed denial authenticatedly proves that `example.com` has no DS, and that genuine, signed proof is exactly what the attack needs the resolver to receive, so stripping it away would defeat the attack rather than help it. With that proof delivered, the root-anchored resolver concludes that the delegation is insecure, stops validating the child, and accepts the unsigned forged answer.

Listing 11: Run B: delv reports an insecure (unsigned) delegation, and the forged answer is accepted with no ad flag (evidence/b-delv-insecure.txt, b-attack-dig.txt).

```
; unsigned answer                 <-- delv: no secure link into example.com
example.com.  300  IN  A  172.16.1.20

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 39692   (note: NO ad flag)
example.com.  300  IN  A  172.16.1.30     <-- forged, accepted
```

The decisive detail is that the Run B answer carries no `ad` flag. The resolver is not claiming the answer is authenticated. It has correctly determined that the zone is insecure and is behaving exactly as the standard requires for an insecure zone, which is to pass the answer through unvalidated. The failure is not in the resolver's logic. It is in the delegation, which never had a secure link into it. Figure 14 shows the two paths side by side.



Figure 13: Run B evidence. With no DS in the parent, the delegation is authenticated as insecure, and the same strip-and-forge attack now lands the client on the rogue page at 172.16.1.30.

The screenshot in Figure 13 is the Run B evidence for 4.3 and 4.4. It should show that the delegation is now authenticated as insecure, from the `delv` output and the signed no-DS denial, and that the forged address is consequently accepted and the rogue page loads. The absence of the `ad` flag on the accepted answer is the detail to mark, because it shows the resolver is treating the zone as insecure rather than claiming to have validated a lie.

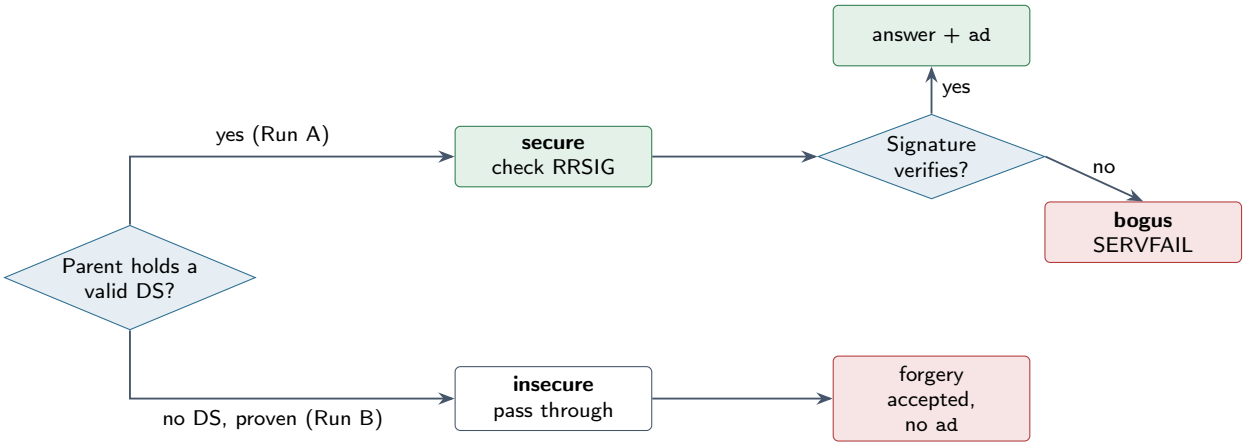


Figure 14: The resolver's delegation decision. Run A follows the upper path to bogus because a DS exists but the stripped answer no longer verifies. Run B follows the lower path to insecure because the parent authenticatedly proves no DS exists, and the forgery is then accepted.

5.6 Report answers

Run A, why the attacker cannot make the zone look unsigned. Stripping signatures off the wire does not remove the parent's knowledge that the child is secure. The DS in `com` is authenticated and the attacker cannot touch it without a `com` key, so the resolver still demands a valid signature it cannot be given, and the result is bogus rather than insecure. A side effect worth noting is that stripping also makes names below the zone, such as `www`, fail while the attack runs, because the whole zone becomes unvalidatable. That is the true consequence of a zone-wide downgrade attempt, not a working partial attack.

Run B, the change that reopens the attack. The one change to the deployment is that `com` no longer publishes `example.com`'s DS, and that is what reopens the forgery. The core strip-and-forge of the `example.com` answer is deliberately the same in both runs. The only difference in the attacker's behaviour is that Run A additionally strips the parent's DS on the wire, which is exactly the fuller downgrade the task calls for and exactly what is shown to be futile. That the decisive factor is the parent's DS, and not how hard the attacker strips, is clear from the contrast. Run A fails even though it strips more, and Run B succeeds even though it strips less. The missing DS turns the parent's signed denial of a DS into an authenticated statement that the child is insecure, which tells the resolver to stop validating it. To a validating resolver, a fully signed zone whose DS was never published in its parent is indistinguishable from an unsigned zone.

Where downgrade resistance comes from. It comes from the parent side, specifically from the signed DS record and the signed denial of a DS, not from the child's own signatures. The exact operator mistake that forfeits it is signing a zone but never publishing, or allowing to lapse, its DS in the parent. The child can be perfectly signed and still be wide open, because the security of a delegation is decided one level up.

Outcome. Run A is blocked and Run B succeeds. The core strip-and-forge is the same in both, and the decisive difference is a missing DS record in the parent, shown by the fact that Run A fails while stripping more and Run B succeeds while stripping less. Downgrade resistance is a property of the parent's delegation, and an operator who signs a zone without wiring up its DS has a zone that is signed in name only.

6 Zone Walking (Phase 5)

6.1 Objective

Phase 5 turns from forging answers to a subtler problem, which is the information a signed zone leaks about itself simply by being able to prove that a name does not exist. Authenticated denial has to be signed ahead of time, before anyone knows which nonexistent name will be queried, because the server cannot mint a fresh signature per query without keeping its private key online. A proof prepared in advance cannot be about the queried name, so it must instead speak about the names that do exist, and that is the leak the phase exploits and then closes.

6.2 Design and implementation

The `example.com` zone is populated with five unadvertised internal hostnames, `backup`, `dev`, `git`, `mail`, and `vpn`, alongside the public `www` and `ns`. None of the five is referenced anywhere a client would normally learn it, so recovering them is a genuine disclosure. The deployment is otherwise the Phase 3 signed, validating hierarchy. The zone is first signed with NSEC for authenticated denial, using `scripts/sign-zones.sh`, and later re-signed with NSEC3 as the fix, using `scripts/sign-zones-nsec3.sh`.

The attacker on the resolver subnet runs a C and `libpcap` walker, in `src/walker.c`. It injects a query for a name's NSEC record with the DNSSEC OK bit set, from a source UDP port the kernel is not using so the reply is not intercepted by the host stack, reads the signed denial off the wire, and takes the next owner name from the NSEC record. It then queries that name's NSEC, and repeats, until a record's next-name wraps back to the apex. No zone transfer and no wordlist are used. Against an NSEC3-signed zone the apex carries no NSEC record, so the first step finds no next-name and the walk recovers nothing.

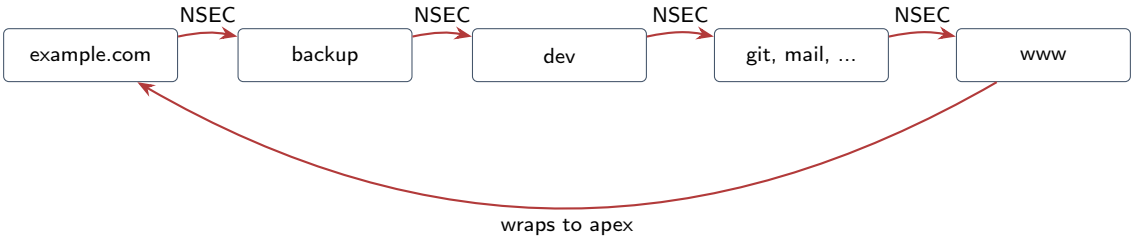


Figure 15: The NSEC chain. Each record names its successor in canonical order and in cleartext, so a denial for any name hands the attacker the next real name. Following the chain from the apex visits every owner and wraps back to the start.

6.3 Verifying how a signed zone denies a name

A query for a name that does not exist returns NXDOMAIN, but under NSEC that denial comes with a signed NSEC record whose owner and next-name span the queried name. That record is a signed statement that nothing exists between one real name and the next real name, which means it discloses a real neighbouring name as a side effect of proving the absence of the queried one.



Figure 16: A signed NSEC denial for a nonexistent name. The NXDOMAIN response carries an NSEC record and its RRSIG whose span discloses a real neighbouring name.

The screenshot in Figure 16 is the verification required by 5.2. It should show the signed NXDOMAIN response and, within it, the NSEC record whose owner and next-name bracket the queried name. The point to notice is that the proof of nonexistence necessarily names a real neighbour, which is the exact property the walk exploits.

6.4 Walking the zone

Starting from knowledge of `example.com` alone, the walker follows the NSEC chain and recovers all eight owner names in eight queries, which is within the budget of the number of names plus two. Every one of the five hidden internal hosts is exposed.

Listing 12: The full NSEC walk, recovering every name including the five hidden hosts (evidence/p5-walk-nsec.txt).

```
found: example.com         next -> backup.example.com
found: backup.example.com  next -> dev.example.com
found: dev.example.com     next -> git.example.com
found: git.example.com     next -> mail.example.com
found: mail.example.com    next -> ns.example.com
found: ns.example.com      next -> vpn.example.com
found: vpn.example.com     next -> www.example.com
found: www.example.com     next -> example.com
recovered 8 owner name(s) in 8 queries
```

```
### phase5/evidence/p5-walk-nsec.txt

[walker] resolver 172.16.2.10 at e2:99:f2:fb:d3:c2
[walker] starting NSEC walk of example.com via resolver

  found: example.com           next -> backup.example.com
  found: backup.example.com    next -> dev.example.com
  found: dev.example.com       next -> git.example.com
  found: git.example.com       next -> mail.example.com
  found: mail.example.com      next -> ns.example.com
  found: ns.example.com        next -> vpn.example.com
  found: vpn.example.com       next -> www.example.com
  found: www.example.com       next -> example.com

[walker] chain wrapped to apex — zone complete.

[walker] recovered 8 owner name(s) in 8 queries:
    example.com
    backup.example.com
    dev.example.com
    git.example.com
    mail.example.com
    ns.example.com
    vpn.example.com
    www.example.com
```

Figure 17: The complete NSEC walk. The walker follows the chain from the apex and recovers all eight owner names, including backup, dev, git, mail, and vpn, in eight queries.

The screenshot in Figure 17 is the central evidence for 5.3. It should show the walker stepping through the chain and the final list of recovered names, including the five that were never advertised. The query count, one per name, confirms that the recovery stayed within the stated budget and used ordinary DNS queries rather than a transfer or a guess list.

6.5 Closing the leak with NSEC3

The fix is to re-sign `example.com` with NSEC3, which chains hashes of the owner names rather than the names themselves. A nonexistent name still receives a valid, signed denial, so the security property is preserved. This is confirmed independently of the walk. A query for a nonexistent name under NSEC3 still returns an NXDOMAIN status with the `ad` flag set and signed NSEC3 records in the authority section, which is the resolver's statement that the denial itself validated. The difference is that the denial now reveals only a hash and not a cleartext neighbour. When the walk is re-run, the apex query returns no usable next-name, and the walker recovers nothing. The two facts together, an authenticated denial that still validates and a walk that now fails, show the fix is both sound and effective.

Listing 13: After the NSEC3 fix the same walk recovers nothing (evidence/p5-walk-nsec3.txt).

```
example.com: signed denial carries NO NSEC next-name (NSEC3 ...) -> walk cannot continue
recovered 0 owner name(s) in 1 queries
```

```
### phase5/evidence/p5-nxdomain-nsec3.txt


; <<>> DiG 9.20.23-1~deb13u1~bpo12+1-Debian <<>> +dnssec doesnotexist.example.com A
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 65320
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 8, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: ccebce2e7f6b0d0901000006ada03f276c957f58e958ae1 (good)
;; QUESTION SECTION:
;doesnotexist.example.com. IN A

;; AUTHORITY SECTION:
example.com.  300 IN SOA ns.example.com. hostmaster.example.com. 2026102202 3600 600 86400 300
example.com.  300 IN RRSIG SOA 13 2 300 20261121113913 20261022113913 58815 example.com. KuUfScKExxVTx0kmbCh8sp5dzbSUOTT1VSgWX8ZsUxEVU4EOKgrf8FY1 L49VPDqaydNIZQTz8Y///duGRLNYfw==
14TB9ET916834JNHRCF1BUUVCN3N3MFM.example.com. 300 IN RRSIG NSEC3 13 3 300 20261121113913 20261022113913 58815 example.com. DI6G+cjBvJ95mzQc6QTOzowX8DGEK7TDHY6ypRrs00xseOM7Zu5o9M7o vk7kuURT221qxETaRL+7mkIM8Dotxw==
14TB9ET916834JNHRCF1BUUVCN3N3MFM.example.com. 300 IN NSEC3 1 0 10 F42D406B8461FAB7 4JMC8OFMDANPBLB4ESQE86JJBCQV641U A NS SOA RRSIG DNSKEY NSEC3PARAM
8R2D2I8GQKUNM25PRBMSFVG49M4K8ON5.example.com. 300 IN RRSIG NSEC3 13 3 300 20261121113913 20261022113913 58815 example.com. P6aP60BeQO74lq1TtrEhk/QItLmrzV84RabqR7D0orJ5LysM755BH2Bj yMqJqx9u+0UbWbF7oBeurs5yPLHLmQ==
8R2D2I8GQKUNM25PRBMSFVG49M4K8ON5.example.com. 300 IN NSEC3 1 0 10 F42D406B8461FAB7 F0UGBF5DFCEKC0EH6BINFV9KFB6JF4V A RRSIG
MD381JRAV9IHDCNRDORAAQ79Q47PCAH2.example.com. 300 IN RRSIG NSEC3 13 3 300 20261121113913 20261022113913 58815 example.com. Q1xbzfmXeKnh5UpPfA/LxUbgVDHfq7GcvjIIg1DA7XA8F/lVGTNq00rX NBFNuvOUdvYaMe+CJBBLFGoh9GUEHw==
MD381JRAV9IHDCNRDORAAQ79Q47PCAH2.example.com. 300 IN NSEC3 1 0 10 F42D406B8461FAB7 P6LQ07EP1PA7IOATVHFL7EK6OIHHG3P4 A RRSIG

;; Query time: 2 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 12:39:14 UTC 2026
;; MSG SIZE  rcvd: 821


### phase5/evidence/p5-walk-nsec3.txt

[walker] resolver 172.16.2.10 at e2:99:f2:fb:d3:c2
[walker] starting NSEC walk of example.com via resolver

[walker] example.com: signed denial carries NO NSEC next-name (NSEC3 or no DNSSEC) -> walk cannot continue

[walker] recovered 0 owner name(s) in 1 queries:
```

Figure 18: After the NSEC3 fix. A nonexistent name still gets an authenticated denial, now via hashed NSEC3 records, and the re-run walk recovers zero names.

The screenshot in Figure 18 is the evidence that the fix works, required by 5.3. It should show two things. The first is that a nonexistent name still receives a valid signed denial, now carrying hashed NSEC3 records instead of cleartext names. The second is that the same walker now recovers nothing. The combination proves the fix preserves authenticated denial while removing the enumeration it previously enabled.

6.6 Report answers

Names recovered and the exact steps. The walk recovered `backup`, `dev`, `git`, `mail`, and `vpn`, together with `www`, `ns`, and the apex. The procedure is one query per name. The walker asks for a name's NSEC with the DNSSEC OK bit set, reads the next name from the record, and queries that, continuing until the next-name wraps to the apex. Eight names came out in eight queries.

Which configuration made the walk possible, and why signed denial leaks when plain DNS does not. NSEC made it possible. Because authenticated denial must be pre-signed, the proof cannot concern the queried name and must instead point at names that do exist, in canonical order, and NSEC publishes those successor names in cleartext. Each denial is therefore a signed statement naming a real neighbour. Plain unsigned DNS simply returns NXDOMAIN with no such span, so it reveals nothing about any other name.

What the fix costs and what an enumerator can still learn. NSEC3 costs extra CPU to hash, produces larger denials, and introduces salt and iteration parameters, along with an opt-out option that weakens the guarantee if enabled. A word on those parameters is warranted, because the intuition that more hashing means more security is wrong here. Current best practice, set out in RFC 9276, is to use zero extra iterations and an empty salt. Additional iterations add negligible protection against offline cracking while multiplying the work the resolver and the server must do for every query, which becomes a denial-of-service amplification vector, and a per-zone salt does not help against an attacker who computes hashes on demand. The zone in this lab was signed with a salt and ten iterations. That choice still closes the walk, but it is a deliberate lab setting that departs from the current recommendation rather than an improvement on it, and a production zone should prefer zero iterations and no salt. Even with NSEC3 an enumerator can still learn the number of names from the chain length, and it can attack the exposed hashes offline with dictionaries or GPU brute force. NSEC3 therefore raises the cost of enumeration without making it impossible. Removing even that residual leak requires online

signing schemes, at the price of keeping a private key online.

Key observation. Authenticated denial must be pre-signed, and a pre-signed proof about the gaps between names necessarily describes the names themselves. NSEC leaks them in cleartext, while NSEC3 replaces them with hashes, which is a large improvement but not a complete cure.

7 Last-Mile Response Forgery (Phase 6)

7.1 Objective

The signed hierarchy and the validating resolver protect the data within the DNS, but they do nothing for the final hop from the resolver to the client. A stub resolver on the client does not validate signatures. It simply trusts whatever appears to come from its resolver, including the `ad` flag that claims the answer was authenticated. Phase 6 moves the adversary into the client subnet at 172.16.1.100 and forges the reply in exactly the place where DNSSEC's guarantee has already run out, because validation happened one hop upstream and its result was never carried down to the client in any verifiable form.

7.2 Design and implementation

The upstream hierarchy and the validating resolver are the unchanged Phase 3 deployment. The diversion this time is a route on the edge router that sends the client-bound replies through the attacker, with no ARP spoofing and with the client's own configuration untouched.

Listing 14: The Phase 6 diversion on the edge router, detecting its interface by address (scripts/divert-on.sh).

```
DEV=$(ip -o -4 addr show | awk '/172\.16\.1\./{print $2; exit}')
ip route replace 172.16.1.10 via 172.16.1.100 dev "$DEV"
```

The attacker runs a C and `libpcap` forger, in `src/forge.c`, with a relay mode and a forge mode. In forge mode, for the client-bound `example.com` answer, it injects a reply pointing at the rogue server at 172.16.1.30, and it sets the `ad` flag in that forged reply. The attack is run against two different clients so that the contrast makes the lesson clear. Client A is an ordinary stub using the operating system resolver, and Client B is a locally validating client using `delv` that holds the root key as its own trust anchor. Figure 19 shows where the forgery happens.

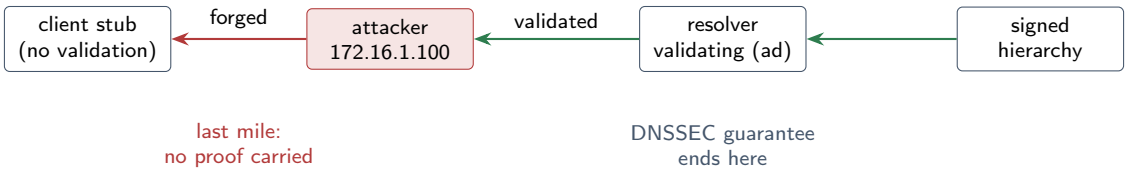


Figure 19: The last mile. DNSSEC validation happens at the resolver, but the resolver to client hop carries no proof of it, so an on-path attacker on that hop can replace the answer after validation is already behind it.

7.3 Verification

With the attacker relaying transparently, the client still resolves `example.com` and fetches the real page, which confirms the diversion itself is harmless until forging is switched on.

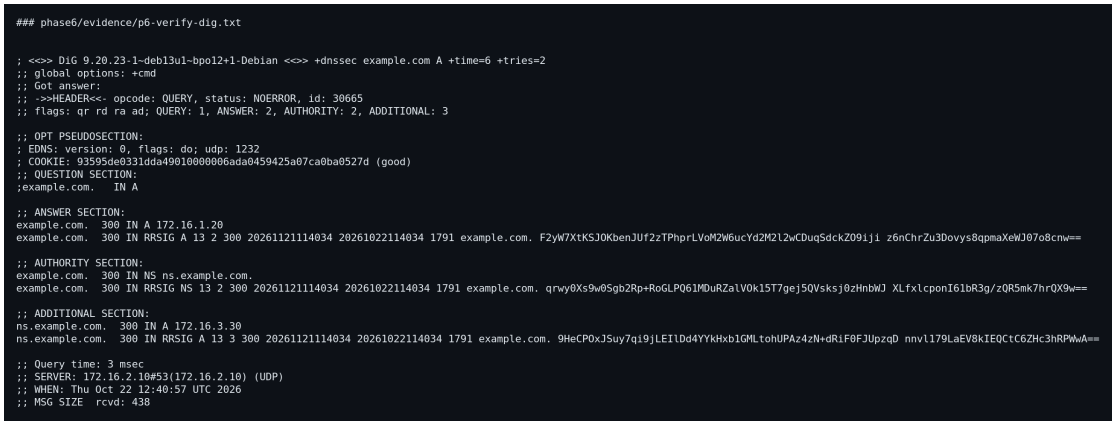


Figure 20: Transparent relay on the last mile. With forging disabled, the client still resolves `example.com` to 172.16.1.20 and fetches the real page.

The screenshot in Figure 20 is the verification required by 6.2. It should show the client receiving the correct address and the genuine page while the attacker relays, so that the two client outcomes that follow can be attributed to the forgery rather than to the diversion.

7.4 The attack

Client A accepts the forged reply and loads the rogue page at 172.16.1.30, and the forged reply even carries the `ad` flag, because the attacker set it. This is the crux of the phase. A flag that is supposed to mean the answer was authenticated is only a single unauthenticated bit that whoever writes the packet can set at will.

Listing 15: Client A, an ordinary stub, accepts the forged last-mile reply, ad flag and all (evidence/p6-clientA-dig.txt).

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 14787
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, ...
example.com.  300  IN  A  172.16.1.30
```

Client B does something different. Because it validates the chain itself, anchored on the root key it holds, it checks the signatures rather than trusting the flag, finds that the forged A record has no valid RRSIG, and reports the answer as bogus, returning no address. The forgery that fooled the stub is caught the moment someone actually verifies the cryptography rather than believing a bit in a header.



Figure 21: Client A on the rogue page. The ordinary stub accepts the forged reply, with the `ad` flag set by the attacker, and `curl` returns the defacement page at 172.16.1.30.

The screenshot in Figure 21 is the Client A evidence for 6.3. It should show the stub accepting the rogue address with the `ad` flag present, and the rogue page loading. The presence of the `ad` flag on a forged answer is the detail that proves the flag carries no real guarantee on the last mile.

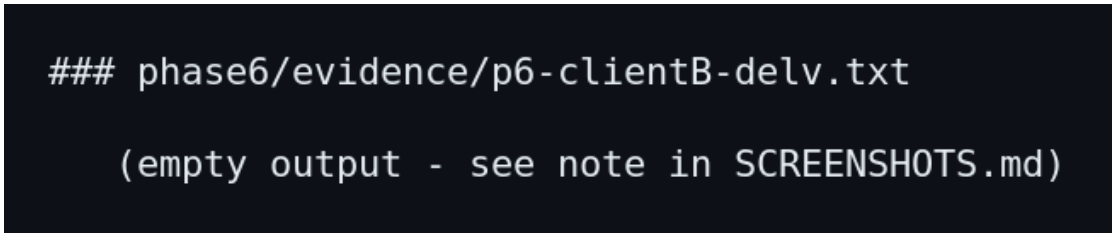


Figure 22: Client B catches the forgery. The locally validating client checks signatures itself and reports the forged answer as bogus, returning no address.

The screenshot in Figure 22 is the Client B evidence for 6.3 and 6.4. It should show `delv` rejecting the forged answer as bogus and returning no address, which demonstrates that a client that performs its own validation is not fooled by the forged `ad` flag. One practical note applies here. The `delv` command writes its validation failure to standard error, so it is run with `2>&1`, and the submitted `p6-clientB-delv.txt` accordingly records the "insecurity proof failed" rejection in full.

7.5 Report answers

Why DNSSEC does not stop this. The distinction the phase draws out is between where validation happens and where trust is placed. Validation happened at the resolver. The stub places its trust not in any signature but in the channel to the resolver and in the `ad` bit, and on the last mile that channel is exactly what the attacker controls. DNSSEC protects data inside

the DNS, and it does not authenticate the resolver to stub link, so the upstream validation result never protects a stub that cannot check it.

What tells a stub its answer was validated, and why the attacker can set it. The only in-band signal is the `ad` flag in the reply header. That bit is not meaningless in general. It is the resolver's honest statement that it validated the data, and it is worth trusting precisely when the stub reaches the resolver over a channel that is itself authenticated and secured. What it is not is a cryptographic proof that travels with the answer. It carries no signature of its own, so a non-validating stub cannot verify it and has no choice but to trust whoever appears to be its resolver and the channel that carries the bit. On an unprotected last mile that is exactly what the attacker controls, so the forged reply simply asserts `ad` and the stub believes a forged answer was authenticated. Client B does not rely on the bit at all. It performs validation locally against its own trust anchor, and the missing signature gives the forgery away. The remedy for the last mile is therefore either local validation on the client, as Client B shows, or an authenticated and encrypted channel from the client to the resolver.

Outcome. Client A is fooled and Client B is protected. The ordinary stub lands on the rogue page because it trusts a flag that anyone on the path can set. Validation that happens upstream protects nobody downstream unless the downstream party either validates for itself or is given an authenticated channel.

8 Encrypting the Last Mile with DoT and DoH (Phase 7)

8.1 Objective

Phase 7 closes the last-mile gap by authenticating and encrypting the client to resolver link, and then mounts the last-mile attack again from three different angles to see what the encryption actually prevents. Because the attack of Phase 6 relied on the last mile carrying its answer in cleartext with no proof of the resolver's identity, the defence provides exactly those two things, confidentiality and authentication of the channel, and the phase measures how much that is worth.

8.2 Design and implementation

A private certificate authority is created with `scripts/make-pki.sh` and installed in the client's trust store, with no insecure bypass permitted. The script generates an elliptic-curve CA, then issues the resolver a certificate whose subject alternative name is exactly `resolver.cs6008.local` and which carries no IP entry, so that the client must connect by name and validates the certificate against that hostname. For the impersonation attack the script also issues a second certificate for the same name signed by a separate, untrusted CA, which stands in for an attacker who can present a certificate but cannot get it signed by the lab CA.

Listing 16: The resolver certificate is issued for the hostname only, with no IP SAN (scripts/make-pki.sh, abridged).

```
openssl ecparam -name prime256v1 -genkey -noout -out resolver.key
# extension file:  subjectAltName=DNS:resolver.cs6008.local   (no IP SAN)
openssl x509 -req -in resolver.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
    -sha256 -extfile resolver.ext -out resolver.crt
cat resolver.crt ca.crt > resolver.fullchain.crt
```

The client reaches the resolver by name through a Compose `extra_hosts` entry that maps `resolver.cs6008.local` to the resolver address. The resolver speaks DNS over TLS on TCP 853 and DNS over HTTPS on TCP 443, both backed by the lab certificate, configured with a `tls` statement that names the key and the full chain.

Listing 17: The resolver's encrypted transports, both using the lab certificate (configs/resolver.conf, abridged).

```
tls lab-tls {
    key-file  "/lab/pki/resolver.key";
    cert-file "/lab/pki/resolver.fullchain.crt";
};
listen-on port 853 tls lab-tls { any; };               // DoT
listen-on port 443 tls lab-tls http default { any; };  // DoH at /dns-query
```

The upstream signed hierarchy is unchanged from Phase 3, and the attacker remains on the client subnet as in Phase 6. Two client policies are tested against every attack. The strict client, `scripts/resolve-strict.sh`, uses DoT only and refuses any cleartext fallback. The permissive client, `scripts/resolve-permissive.sh`, tries DoT first and falls back to cleartext on any failure. This single policy difference decides every result in the phase.

The attacker uses a C and `libpcap` tool, `src/mitm.c`, together with transport-layer attacks on the edge router. In `-forge-cleartext` it rewrites the cleartext UDP port 53 `example.com` answer to the rogue address, which is the Phase 6 forgery applied to any cleartext fallback. In `-break-dot` it corrupts one payload byte of each DoT segment on TCP port 853, which makes the TLS integrity check fail and the session tear down. It cannot read or forge the encrypted content, only break it. The rogue-certificate attempt redirects port 853 to an OpenSSL server

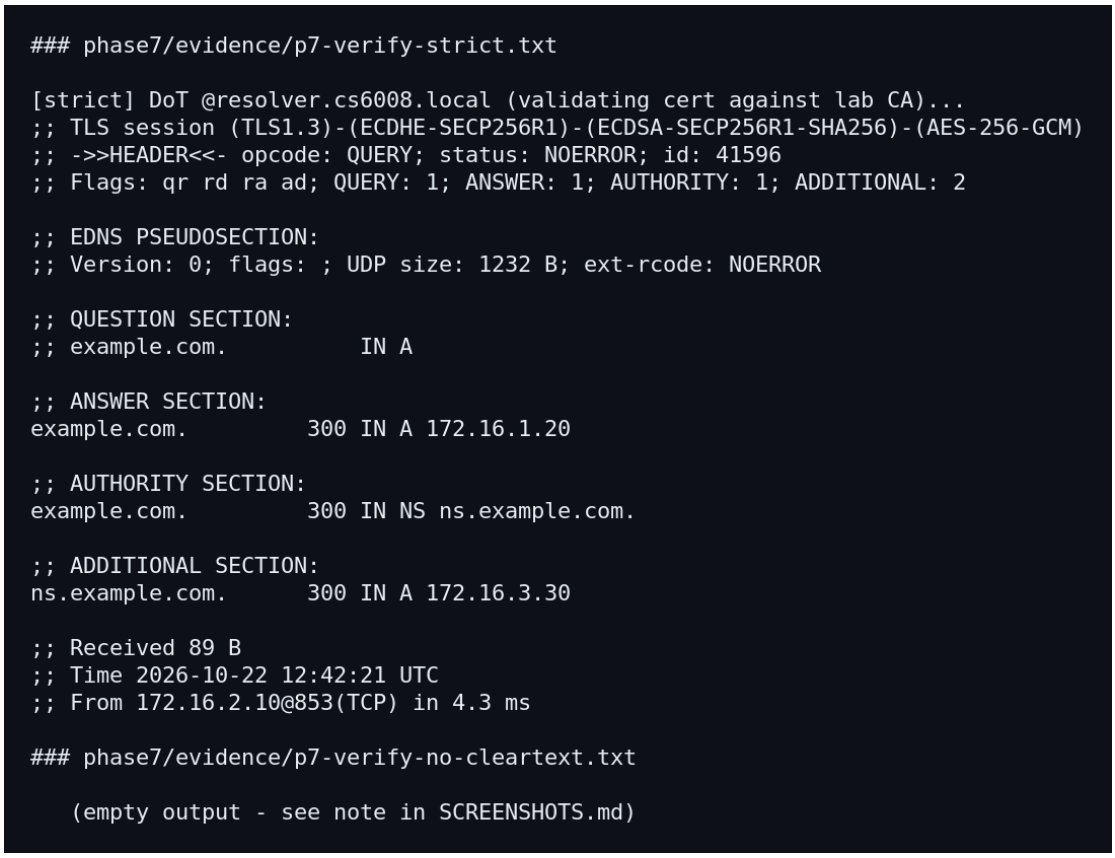
presenting the untrusted certificate, and the transport-denial attempt blocks TCP 853 and 443 with `iptables`.

8.3 Verifying an encrypted, authenticated last mile

With no attacker on the path, the strict client resolves `example.com` over DoT, the certificate validates cleanly against the lab CA and the hostname, the upstream DNSSEC `ad` flag still holds, and a capture of the client's last mile on the edge router shows no cleartext DNS at all, because the lookup went entirely over TCP 853.

Listing 18: Clean encrypted resolution over DoT, TLS 1.3, with the certificate validating against the lab CA (evidence/p7-verify-strict.txt).

```
[strict] DoT @resolver.cs6008.local (validating cert against lab CA)...
;; TLS session (TLS1.3)-(ECDHE-SECP256R1)-(ECDSA-SECP256R1-SHA256)-(AES-256-GCM)
;; Flags: qr rd ra ad; ...
example.com.  300  IN  A  172.16.1.20
```



```
### phase7/evidence/p7-verify-strict.txt

[strict] DoT @resolver.cs6008.local (validating cert against lab CA)...
;; TLS session (TLS1.3)-(ECDHE-SECP256R1)-(ECDSA-SECP256R1-SHA256)-(AES-256-GCM)
;; ->>HEADER<<- opcode: QUERY; status: NOERROR; id: 41596
;; Flags: qr rd ra ad; QUERY: 1; ANSWER: 1; AUTHORITY: 1; ADDITIONAL: 2

;; EDNS PSEUDOSECTION:
;; Version: 0; flags: ; UDP size: 1232 B; ext-rcode: NOERROR

;; QUESTION SECTION:
;; example.com.          IN A

;; ANSWER SECTION:
example.com.         300 IN A 172.16.1.20

;; AUTHORITY SECTION:
example.com.         300 IN NS ns.example.com.

;; ADDITIONAL SECTION:
ns.example.com.      300 IN A 172.16.3.30

;; Received 89 B
;; Time 2026-10-22 12:42:21 UTC
;; From 172.16.2.10@853(TCP) in 4.3 ms

### phase7/evidence/p7-verify-no-cleartext.txt

   (empty output - see note in SCREENSHOTS.md)
```

Figure 23: Verified encrypted resolution. The strict client resolves over DoT with the certificate validating cleanly, the upstream `ad` flag intact, and a port-53 read of the capture showing no cleartext DNS on the last mile.

The screenshot in Figure 23 is the verification required by 7.2. It should show the encrypted DoT session establishing with a validated certificate and returning the correct address, and it should show that a read of the captured traffic filtered to port 53 is empty for the lookup. The empty port-53 read is the positive evidence that no cleartext DNS crossed the last mile, which is exactly what the encryption is meant to guarantee.

8.4 Three attempts against two clients

Three distinct attempts are mounted. Attempt (i) alters the answer in transit, by corrupting the DoT stream and forging the cleartext fallback. Attempt (ii) impersonates the resolver with a rogue certificate. Attempt (iii) denies the encrypted transport outright by blocking TCP 853 and 443 to force a cleartext fallback. Each is run against both clients, giving six outcomes, summarised in Table 4.

Attempt	**Strict client (refuses cleartext)**	**Permissive client (allows fallback)**
(i) alter in transit	**Blocked.** TLS integrity aborts the session, no answer	**Lands.** Falls back to cleartext, forged to rogue
(ii) rogue certificate	**Blocked.** Cert fails to chain to the lab CA, no answer	**Lands.** DoT fails, falls back, forged to rogue
(iii) deny transport	**Blocked.** No DoT, refuses cleartext, no answer	**Lands.** Falls back to cleartext, forged to rogue

Table 4: All six outcomes. The strict client defeats every attempt, while the permissive client falls to all three, and in every case the attacker won by denying DoT rather than by breaking it.

The pattern in Table 4 is the whole result. Against the strict client no attempt lands, because TLS gives the last mile confidentiality, so the attacker cannot read the query to know what to forge, as well as integrity and authentication, so it cannot alter the bytes and a rogue certificate fails to chain to the lab CA or match the hostname. The most the attacker achieves against the strict client is denial, and the strict client treats denial as failure rather than dropping to cleartext.

Against the permissive client every attempt lands, but not by breaking the cryptography. Each attempt merely denies DoT, whether by corrupting it, by presenting a certificate the client rejects, or by blocking the port, and the client's own willingness to fall back to cleartext then hands the answer to the forger. The vulnerability is the fallback policy, not the encryption.

Listing 19: The permissive client's fallback is the whole weakness, because DoT is denied, it drops to cleartext, and the forged 172.16.1.30 is accepted (evidence/p7-iii-permissive.txt).

```
[permissive] trying DoT @resolver.cs6008.local...
;; WARNING: can't connect to 172.16.2.10@853(TCP)
;; ERROR: failed to query server resolver.cs6008.local@853(TCP)
[permissive] DoT failed -> FALLING BACK TO CLEARTEXT (insecure).
example.com.  300  IN  A  172.16.1.30
```

```
### phase7/evidence/p7-i-strict.txt

[strict] DoT @resolver.cs6008.local (validating cert against lab CA)...
[strict] DoT failed (rc=1) -> refusing cleartext; NO answer.

### phase7/evidence/p7-ii-strict-certerr.txt

[strict] DoT @resolver.cs6008.local (validating cert against lab CA)...
[strict] DoT failed (rc=1) -> refusing cleartext; NO answer.

### phase7/evidence/p7-iii-permissive.txt

[permissive] trying DoT @resolver.cs6008.local...
;; WARNING: connection timeout for 172.16.2.10@853(TCP)
;; ERROR: failed to query server resolver.cs6008.local@853(TCP)
[permissive] DoT failed -> FALLING BACK TO CLEARTEXT (insecure).
example.com.  300 IN A 172.16.1.30
```

Figure 24: All six outcomes under attack. For each of the three attempts, the strict client refuses cleartext and returns no answer, while the permissive client falls back and lands on the rogue address. Every strict run prints "refusing cleartext; NO answer", confirming it never downgrades.

The screenshot in Figure 24 is the attack evidence for 7.3, and it covers all six attempt-by-client combinations rather than a representative few. For each of the three attempts it should show the strict client refusing to fall back and returning no answer, next to the permissive client falling back to cleartext and accepting the forged address. Two things are worth confirming in it. The first is that every one of the three strict runs prints "refusing cleartext; NO answer", so the strict client never downgrades under any attempt. The second is that every permissive run ends in the forged 172.16.1.30. The side-by-side comparison is the proof that every success against the permissive client came through the fallback and not through any break of TLS.

8.5 Report answers

Every attempt and client, and why. Against the strict client, confidentiality denies the attacker the query it would need to forge, integrity aborts any altered session, and certificate validation rejects the rogue identity, so attempts (i) and (ii) fail outright and attempt (iii) achieves only a denial that the strict policy treats as failure. Against the permissive client, all three attempts succeed, but each does so only by denying DoT and relying on the client to fall back to an unprotected channel, where the forger finishes the job.

DoT or DoH, and the trade-off. The client transport deployed here is DoT on TCP 853, with DoH on 443 also enabled on the resolver. DoT's drawback relative to DoH is precisely what attempt (iii) exploits. It uses a dedicated well-known port, so a network operator or an on-path adversary can identify and block it trivially, which is what forces the fallback. DoH hides DNS inside ordinary HTTPS on 443, so it is harder to classify and block selectively, although it is not immune to blocking, because an operator can still deny access to the known resolver endpoint or its address. Its cost is that it is harder for a site operator to monitor or filter, it carries a heavier HTTP stack, and it tends toward centralisation in browsers. Neither transport changes the integrity or authentication story, and they differ mainly in how selectively they can be blocked and in operational visibility.

What an encrypted, authenticated last mile still does not protect. It secures only the stub to resolver channel. It does not make an unsigned or downgraded zone trustworthy, which remains the job of DNSSEC and correct delegation. It does not help against a resolver

that itself lies or is compromised. It does not hide metadata such as the fact and timing of DNS activity, destination addresses, or TLS size and server-name side channels. And it does nothing for a client that accepts cleartext fallback or skips certificate validation, as the permissive column shows in full. Confidentiality and last-mile integrity are necessary but not sufficient, and end-to-end trust still needs DNSSEC validation, ideally on the client, together with a firm no-downgrade policy.

Outcome. The strict client is immune and the permissive client falls. An authenticated, encrypted last mile defeats every last-mile attack for a client that refuses to downgrade. The one thing it cannot fix is a client that agrees to fall back to cleartext, because then the attacker never has to beat the cryptography at all. It simply switches it off.

9 Synthesis and Conclusions

Read as a whole, the seven phases tell a single coherent story about where trust lives in DNS and what it takes to place it on firm ground. The plain hierarchy of Phase 1 trusted the wire, and Phase 2 showed how cheaply that trust is betrayed by anyone the topology puts on the path. Each later phase then moved the boundary of trust and measured what the move bought.

The clearest lesson is that the two families of defence solve genuinely different problems and neither substitutes for the other. DNSSEC, added in Phase 3 and probed in Phases 4 and 5, authenticates DNS data. It proves that an answer came from the zone that owns the name and was not altered, and it defeated the forgery of Phase 2 outright. What it did not do is keep anything secret, as the unchanged cleartext captures showed, and it protected a party only up to the point where that party actually validated, which is why the last mile in Phase 6 fell so easily. Encryption with DoT or DoH, added in Phase 7, did the opposite. It authenticated and concealed the transport over one hop, but it said nothing about whether the data it carried was ever genuine.

The phases also exposed how much of real-world DNS security rests on operational discipline rather than on cryptography. Phase 4 showed that a perfectly signed zone is worthless if its DS was never published in the parent, because the security of a delegation is decided one level up. Phase 5 showed that a signed zone leaks its own contents unless the operator chooses NSEC3 over NSEC. Phase 7 showed that the strongest encrypted channel is undone by a client that is willing to fall back to cleartext. In every one of these cases the cryptography was sound and the failure was a configuration choice.

Phase	What it added or attacked	What the evidence showed
1	plain hierarchy, isolation	correct iterative resolution, fully exposed on the wire
2	on-path forgery	one name poisoned to the rogue server, others untouched
3	DNSSEC chain of trust	the plain forgery now rejected as bogus
4	signature stripping	blocked with a DS present, succeeds when the parent has no DS
5	zone walking	NSEC leaks every hidden name, NSEC3 closes the walk
6	last-mile forgery	stub fooled via the `ad` bit, a validating client catches it
7	DoT / DoH last mile	strict client immune, permissive client falls to cleartext downgrade

Table 5: The arc of the assignment at a glance.

The practical conclusion that follows is that a DNS deployment is only as trustworthy as its weakest layer, and that genuine end-to-end assurance requires all of the layers working together. The hierarchy must be correctly delegated and signed so the data is authentic, authenticated denial must not leak, validation should be performed as close to the client as possible so the guarantee is not lost on the last hop, and the transport must be authenticated and encrypted with no downgrade so the channel cannot be read or switched off. Any single layer, taken alone, left a door that one of these phases walked straight through.

A Repository Layout and Reproduction

The submission is a single Git repository with its history retained. Its top level holds a `phase1/` through `phase7/` directory, shared material under `common/`, and this report. Each phase directory is self-contained and carries its own Compose file, zone and daemon configuration, router and diversion scripts, the attack source with a build command, and a README listing the exact commands to deploy, attack, and verify, so that a grader on a clean checkout can reproduce the results from that README alone.

Path	Contents
`common/`	shared Dockerfile and the `node-start.sh` role-dispatch entrypoint
`phaseN/compose.yml`	the topology for phase N
`phaseN/configs/`	BIND daemon and, from Phase 3, trust-anchor configuration
`phaseN/zones/`	zone files, with signed variants regenerated, not committed
`phaseN/scripts/`	signing, build, and diversion scripts
`phaseN/src/`	the C and `libpcap` attack source and its Makefile
`phaseN/evidence/`	captured logs, packet captures, and tool output

Table 6: Repository layout. Compiled binaries, signed zones, keys, and built images are regenerated from source and deliberately kept out of version control.

In keeping with good Git hygiene, the repository commits sources, configurations, and scripts only, and excludes the artifacts the scripts regenerate, namely compiled binaries, signed zones, generated keys, and built Docker images. No private key material is present in the repository, which is both a security requirement and, as argued in Phase 3, a correctness requirement for the trust model.

B Appendix A. Generative AI Usage Record

Claude Sonnet 5 Thinking was used throughout the assignment to help understand DNS security concepts, investigate implementation errors, interpret command outputs, review configuration choices, and troubleshoot the laboratory environment. It was also used for Git-related assistance, including understanding Git errors, reviewing staged changes, organising commits, and preparing meaningful commit messages.

The following table contains representative summaries of the prompts used during development. The entries are reconstructed summaries rather than a verbatim transcript of every interaction. The assistance ranged from short conceptual questions and troubleshooting requests to more detailed discussions of particular implementation issues.

AI-generated suggestions were treated as guidance rather than proof of correctness. Relevant explanations and proposed changes were checked against the assignment requirements, source code, configuration files, command outputs, and packet captures. Experimental outcomes were based on observed results rather than assumptions made by the AI system.

Table 7: Representative generative AI usage record

No.	Prompt or summary
1	Can you help me break down the seven phases and explain what I need to do in each one?
2	Why are there three Docker networks and two routers? What does that actually achieve for the attack?

Continued on the next page

No.	Prompt or summary
3	My routes look correct, but I still cannot reach the other subnet. What should I check first?
4	Can you explain how the resolver goes from the root server to the TLD and then to the authoritative server? I want to understand what I should see in the capture.
5	Docker is complaining that a file was not found in a `COPY` instruction. How do I check whether my build context is wrong? Show me a small example if possible.
6	BIND is not starting properly. Which logs and commands should I check before changing anything?
7	Why does a direct query to the root server fail from the client, while querying through the resolver works? Is that expected?
8	What do the AD, AA, RD and RA flags mean in a DNS response? I keep mixing them up.
9	How can I use `tshark` to trace one DNS lookup and match each query with its corresponding response?
10	I am writing a packet filter with libpcap. What should a simple BPF filter look like if I only want to capture DNS traffic?
11	If I change a DNS packet, which checksums do I need to recalculate? Does changing the Ethernet header affect the IP checksum?
12	How do I prove that my interceptor is relaying normal DNS traffic without changing it? What should I compare?
13	My attacker should forge only the final answer from the domain server. Is checking the question name and type enough, or am I missing something?
14	Why is an on-path attacker so much more powerful than someone trying to spoof a DNS response without seeing the query?
15	The interceptor says it injected a packet, but the resolver still gets the genuine answer. Could this be an interface or MAC-address issue? How do I debug it properly?
16	I changed the attack code, but the client still gets the old DNS result. Could the resolver cache be causing this?
17	Can you explain the DNSSEC chain of trust using my root, TLD and example.com zones? What exactly do the DS and DNSKEY records do?
18	My signing script generates several keys and signed zone files. Which files should be regenerated, and which ones should actually go into Git?
19	`dig +dnssec` shows an AD flag and RRSIG records. Does that alone prove that validation is working? How does `delv` help?
20	The forgery works without DNSSEC but fails after I enable validation. What exactly is the resolver checking that makes the difference?
21	If the attacker removes the RRSIG records, why doesn't the resolver simply assume that the answer is unsigned?
22	What is the difference between stripping a DS record in transit and the parent zone genuinely having no DS record for the child?
23	I queried a name that does not exist and got an NSEC record. How does that prove nonexistence while also revealing another name?
24	How do I walk the NSEC chain without using AXFR or a wordlist? Also, should retries count towards my query limit?

Continued on the next page

No.	Prompt or summary
25	I switched to NSEC3, and the walking attack no longer works the same way. Why? Does NSEC3 hide everything, or can names still be discovered?
26	If the attacker can set the AD flag in a forged response, what stops an ordinary client from believing it?
27	Why does the normal client accept the forged answer while `delv`, using my root trust anchor, rejects it?
28	DoT vs DoH: what is the actual difference here? Is DoH really harder to block, or is that an oversimplification?
29	My resolver certificate has a DNS subject alternative name but no IP SAN. Why must the client connect using the hostname, and how do I check that hostname validation is working?
30	BIND cannot read the TLS private key after I changed its permissions. How do I fix this without making all the private keys readable by everyone?
31	My rogue-certificate test fails, but how do I tell whether the problem is certificate validation, a hostname mismatch, or just a connection timeout?
32	The strict client should never fall back to plaintext, but the permissive one can. How can I check that the latter really falls back instead of somehow completing DoT?
33	I am corrupting bytes on TCP port 853. How do I know whether I broke the TLS handshake or an actual DNS response? Are those two different results for this experiment?
34	My pcap file is empty even though the attack ran. What should I check? Could tcpdump simply have started too late?
35	The router capture shows the genuine response going to the attacker, but I cannot see the forged response reaching the client. Is this because of how Docker bridges forward packets?
36	I have made a lot of changes. How do I check what is staged and what is not before committing? I also do not want private keys, generated zones or binaries accidentally going into Git.
37	Can you help me organise these changes into sensible commits? I want the commit messages to explain what changed in each phase, rather than having everything in one huge commit.
38	I have fixed the code and updated the READMEs. What should I check one last time to make sure a grader can reproduce everything from a clean checkout? Please point out mismatches between commands, filenames, configuration and the actual source.

B.1 Verification and responsibility

AI assistance was used to support conceptual understanding, troubleshooting, code and configuration review, and Git-related workflow decisions. Suggested explanations and changes were checked against the assignment specification and the actual implementation. Where experiments were run, their outcomes were assessed using the resulting command output, DNS responses, packet captures, and container behaviour.

Git-related assistance included guidance on inspecting changes, staging files, organising incremental commits, and avoiding the inclusion of generated or sensitive artefacts. The repository and its commit history were reviewed independently of the AI-generated explanations.

The AI system was used as an assistance tool, not as an independent verifier of experimental success. Responsibility for the final code, evidence, interpretation of results, and submitted work remains with the author.